



**Politecnico
di Torino**

Politecnico di Torino

Corso di Laurea Magistrale

A.a. 2026/2027

Large Language Models

Docenti:

Flavio Giobergia
Riccardo Coppola

Studente:

Cristiano Berardo

Indice

I	Introduction to LLM	1
0.1	What is a Language Model (LM)?	2
0.1.1	Formal definition for language models	2
0.2	Simple LMs - n-gram models	2
0.3	Estimation & generation	2
0.4	Generating new sentences	2
0.5	Limitation of n-grams	2
0.6	Bigger and better Language Models	3
1	Introduction to deep learning	4
1.1	The perceptron	4
1.2	The perceptron, in 2D	4
1.3	Activation functions	4
1.4	1. Enforce properties on perceptron's output	5
1.4.1	Adding some perceptrons	5
1.4.2	and adding other layers!	5
1.5	2. Introduce non-linearities in the model	5
1.6	Multi-layer perceptron models	5
1.7	Activation functions for classification models	5
1.8	Binary classification	6
1.9	Multi-class classification	6
1.10	Activation functions for regression models	6
1.11	Defining weights (parameters)	6
1.12	Linear regression	6
1.13	More complex losses/model	6
1.13.1	Limitation of gradient descent	7
1.14	Backpropagation	7
1.14.1	Using the chain rule	7
1.14.2	Computational graph	7
1.15	Loss functions	8
1.16	Loss functions	8
2	Vector Embedding	9
2.1	One-Hot encoding	9
2.2	Distributed representation	9
2.3	In terms of matrices	9
2.4	Limitation of WordToVec	10
2.5	FastText	10
2.6	Visualizations - Semantic meanings	10
2.7	Inside an RNN	10
2.7.1	Weight sharing	10
2.8	Encoder-decoder architecture	10
3	Recurrent Neural Networks	11
3.1	Recurrent Neural Networks - RNN	11

4	Transformers	12
4.1	Simplified architecture	12
4.2	Generation the first token	12
4.3	Tokenization	13
4.4	Byte-Pair Encoding - desiderata	14
	4.4.1 BPE results	14
4.5	Some extra tokens	15
4.6	Positional encoding	15
4.7	Sinusoidal Positional encoding	15

Parte I

Introduction to LLM

0.1 What is a Language Model (LM)?

A Language Model is a probabilistic model of a natural language. It produce a probability for any sequence of words.

The probability over all possible sentences in a language sums to 1. There are region of high density in the space of all possible sentences.

0.1.1 Formal definition for language models

$$P(w_1, w_2, \dots, w_n) = \prod_{t=1}^T P(w_t | w_1, w_2, \dots, w_{t-1})$$

0.2 Simple LMs - n-gram models

We need to estimate the probability of a word given the previous words. A simplifying assumption, Markov assumption, is that:

The next word w_t depends only on a small windows of $n-1$ preceding words called the **context**.
For $n = 2$, the next word only depends on the previous word, called a **bigram** model.

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_1) \approx P(w_t | w_{t-1})$$

0.3 Estimation & generation

We can use a language model to estimate probabilities for new sentences. Generate a new sentence based on known probabilities.

- Starting from a given word, we can compute the probability of having any other word following it
- Then, we choose one word based on this probability distribution
- We add the new word to the already available sequence, and generate the next one
- We call this, autoregressive generation

Example $P(\text{the, cat, chased, the, cheese}) = P(\text{the}) \times P(\text{cat}|\text{the}) \times P(\text{chased}|\text{the, cat}) \times P(\text{the}|\text{the, cat, chased}) \times P(\text{cheese}|\text{the, cat, chased, the})$

We can use the Markov assumption to simplify the computation of the probabilities.

$$P(\text{the, cat, chased, the, cheese}) \approx P(\text{the}) \times P(\text{cat}|\text{the}) \times P(\text{chased}|\text{cat}) \times P(\text{the}|\text{chased}) \times P(\text{cheese}|\text{the})$$

Let's assume $P(\text{the}) = 1$

$$P(\text{the, cat, chased, the, cheese}) \approx 1 \times \frac{1}{4} \times 1 \times 1 \times \frac{1}{4} = \frac{1}{16}$$

0.4 Generating new sentences

0.5 Limitation of n-grams

1. **Data sparsity:** As n increases, the number of possible n-grams grows exponentially. Bigram, V words, the table size is V^2 . n-grams, V words, the table size is V^n . A small vocabulary or a small context size make the LM quite useless
2. **Context limitation:** Only local context (previous $n-1$ words) are used to choose. Longer sentences may require remembering what happened "early on"
3. **Lack of semantics:** Similar words are treated in the same way as completely different ones

The language models are not a new concept. In the '90s, researchers tried to build LMs based on existing techniques and computing capabilities. With varying results... Mostly bad.

0.6 Bigger and better Language Models

We can see $P(w_t|w_{t-1}, w_{t-2}, \dots, w_1)$ as a classification function.

"Given an **input**, the previous words in the sentence, predict the **next word**, class."

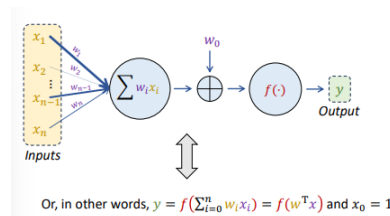
Capitolo 1

Introduction to deep learning

1.1 The perceptron

The perceptron is the simplest unit of neural networks. It takes an input and **multiplies** it by a weight. Then, it sums all the weighted inputs and applies a non-linear function to produce an output.

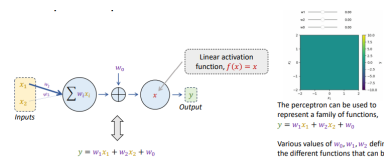
$$y = f(w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n)$$



Where:

- y is the output of the perceptron
- f represents a non-linear “activation” function
- w_i and w_0 are the weights and bias which are learned during training
- $x = (x_1, x_2, \dots, x_n)$ are the inputs to the perceptron

1.2 The perceptron, in 2D

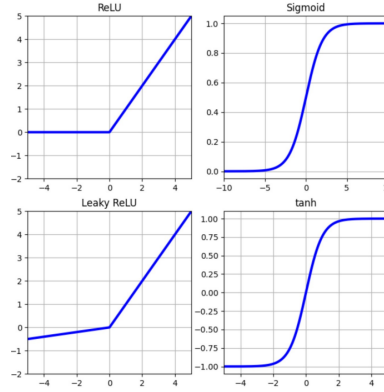


1.3 Activation functions

Activation functions are used for two main reasons:

- Enforce properties on perceptron's output. E.g. sigmoid -> binds output to $[0, 1]$ range
- Introduce non-linearities in the model + some others (faster convergence, sparsity, ...)

Common activation functions are: ReLU, Sigmoid, Leaky ReLU, Tanh, Softmax, Linear, GeLU



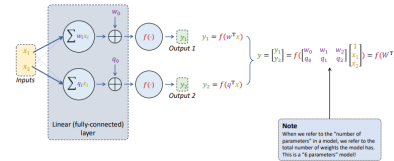
1.4 1. Enforce properties on perceptron's output

Binary classification problem: separate positive ($\star$) and negative ($\circ$) samples. For a point $x \in \mathbb{R}^2$, the perceptron can predict $p(\star|x)$. For a binary case this implies that $p(\circ|x) = 1 - p(\star|x)$.

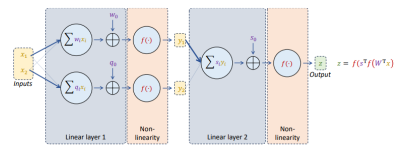
The Sigmoid maps any value in $\mathbb{R}$ to the range $[0, 1]$. The output is squashed between 0 and 1.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

1.4.1 Adding some perceptrons



1.4.2 and adding other layers!



1.5 2. Introduce non-linearities in the model

This is the second reason why we need activation functions.

If $f(x) = x$ (i.e., no non-linearity is added) we get $z = s^T W^T x$

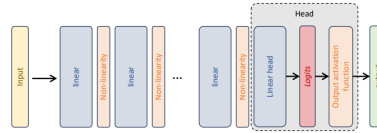
1.6 Multi-layer perceptron models

1.7 Activation functions for classification models

As argued, activation functions can be used to enforce properties on the model's output

In classification problems, the output before the final activation is treated as unnormalized probabilities **logits unit**.

We still need a step to convert logits into valid probabilities



1.8 Binary classification

1.9 Multi-class classification

The **softmax** function produce a probability of class (e.g. class 1 (cat), class 2 (dog)), it gives us only values between 0 and 1

$$P(c_i|x) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$



1.10 Activation functions for regression models

1.11 Defining weights (parameters)

So far, we assumed all weights and biases (let's call them θ) to be known. **But, we still need to figure out how we find them!**

We pick a function, onjective, or loss, $\mathcal{L}(\theta)$, that we want to minimize.

It looks at the model, the data are fixed, and give us a number, high number means bad model, low number means good model.

Example:

$$\mathcal{L}(\theta) = MSE(\theta) = \frac{1}{n} \sum (y_i - \theta^T x_i)^2$$

Then, we pick θ that minimizes it

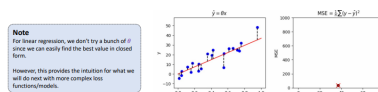
1.12 Linear regression

For simple models, we can find the optimal weights in closed form

$$\frac{\partial \mathcal{L}(\theta)}{\partial \theta} = \frac{\partial MSE(\theta)}{\partial \theta} = 0$$

Quadratic in θ , can be solved easily!

Or, we can evaluate the loss function for a bunch of θ 's, and find the “best” one



1.13 More complex losses/model

In more complex loss functions/models, we may not be able to solve the problem in a closed form. But we can evaluate $\mathcal{L}(\theta)$ for various values of θ .

We can iteratively update θ to reach a local minimum. We can start from a random value θ , then:

- We start from a random value θ , then

- we “move around” according to “some policy”

Move around = update θ incrementally, based on its current value. The new value of θ at any step depends on the previous step's value: $\theta_{t+1} := \theta_t + \text{update}$

Some policy = we take a small step in the direction where the function decreases locally. i.e. in the **opposite** direction of the **gradient**. $\theta_{t+1} := \theta_t - \alpha \nabla_{\theta} \mathcal{L}(\theta_t)$.

For 1-dimensional θ , we have $\theta_{t+1} := \theta_t - \alpha \frac{\partial \mathcal{L}(\theta)}{\partial \theta}$ α : learning rate, controls the “size” of the step (10^{-3} , 10^{-4} , ...)

If the derivative is negative, i move to the right, when the derivative is positive i move to the left and i get the bottom of the function.

Gradient Descent!

1.13.1 Limitation of gradient descent

GD is sensitive to weight initialization. Different initializations can lead to different solutions!. GD can get stuck in local minima

Various solutions to help prevent local minima:

- Adding momentum
- Adaptive learning rates, how to regulate the step
- Learning rate schedules

1.14 Backpropagation

How to compute the gradient of a loss function with respect to the weights of a neural network?

So far, we assumed we were able to compute $\nabla_{\theta} \mathcal{L}(\theta)$.

We need some automated way to compute gradients.

Now we introduce the **backpropagation**, to compute the gradient of the loss w.r.t. any weight! Backpropagation is just a fancy word for “using the chain rule”

1.14.1 Using the chain rule

We use the chain rule from calculus

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial x}$$

Other notation could be $f(g(x))' = f'(g(x)) \cdot g'(x)$

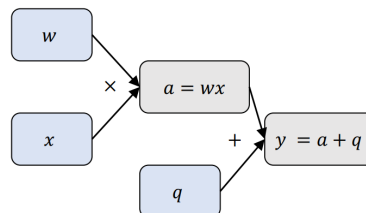
And apply it from the end of the computational graph, backwards. Hence the name, backpropagation.

1.14.2 Computational graph

A computational graph is a directed graph

- Each node corresponds to an operation
- Each edge represents the flow of data between nodes

For instance, we may want to compute $y = wx + b$



We start from three variables, w , x and y . The computational graph performs one operation at a time.

- First, compute the intermediate variable $a = wx$
- Then, compute the output variable $z = a + y = wx + y$

Backpropagation example Let's say:

- Our dataset has one point (x, y)
- Our (weird) model has two parameters, θ_1 and θ_2 , and predicts $\theta_1\theta_2x$
- Our loss function is $L = (\theta_1\theta_2x - y)^2$

We build a computational graph with all operations and intermediate variables.

- $a = \theta_1\theta_2$
- $b = ax = \theta_1\theta_2x$
- $c = b - y = ax - y = \theta_1\theta_2x - y$
- $L = c^2 = (b - y)^2 = (ax - y)^2 = (\theta_1\theta_2x - y)^2$

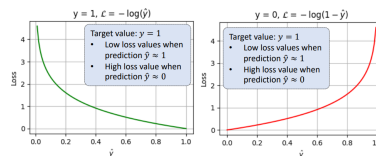
1.15 Loss functions

What loss function should I use?

- Regression: Mean Squared Error (MSE), Mean Absolute Error (MAE)
- Binary: Binary Cross Entropy (BCE), the ground truth is $y = 0, 1$ and the predicted value is $\hat{y} = \text{model}(x) \in [0, 1]$

y , the ground truth, acts as a "selector" of the loss term to be applied.

- $y = 1 \rightarrow \mathcal{L} = -\log(\hat{y})$
- $y = 0 \rightarrow \mathcal{L} = -\log(1 - \hat{y})$



1.16 Loss functions

Multi-class classification

- Cross-Entropy
- Generalization to multiple classes of BCE
- $y_i = 1$ when ground truth is the i th class, 0 otherwise
- y_i plays the same "selector" mechanism as in BCE

$$\mathcal{L} = - \sum_i y_i \log(\hat{y}_i)$$

Capitolo 2

Vector Embedding

Transformers build in 2017 after the famous Google's paper "attention is all you need". Embeddings: advanced representations to represent words/tokens in a vector in a dense way with limited numbers of dimensions 300 dims. We would like to have: semantic meaning of a word relationship among words, words with similar connections are linked via similar transformers

dog	→	[1 0 0 0 0]
cat	→	[0 1 0 0 0]
fish	→	[0 0 1 0 0]
pen	→	[0 0 0 1 0]
pencil	→	[0 0 0 0 1]

2.1 One-Hot encoding

One-hot encoding was used before embeddings. Vocabulary of 5 words with 5 dimension vector and set 0s and only one 1 each word. Disadvantages: This approach is sparse, dimensions = words count, not feasible. Not preservation of similarity and no relationship among words. Vectors are orthogonal.

2.2 Distributed representation

We distribute the semantic word across all the dimension. But how do we figure it the vector? We use NN model to learn these words and create these vectors. E.g. I use a to write the essay. We ask the model to predict the blank, this encourages the model to figure it out, starting from the surrounded words, context. Can we figure out the probability of each vocabulary, based on the context? Some steps to

- assign a vector for each word in the context, at beginning is random number
- Take all of these vectors and aggregate all vectors and sum them up (**context vector**)

Context vector and vector of all possible words.

Cosine similarity = cross entropy → gradient descent → backpropagation → update the vectors to make the loss function go down. In this is repeated for millions of sentences. Pencil or pen now are similar and connected, semantic related, now two words can have same number, but it is fine.

2.3 In terms of matrices

We need to compute a matrix multiplication. All vectors for input words can be stacked into matrix.

This was presented in a paper of 2000. Then in the 2014, thirty years later.

2.4 Limitation of WordToVec

- Cannot manage words are not in the vocabulary, OOV (out of vocabulary)
- This vector are not contextualized

After the tranign, the vector is the same, and during the inference some word depends on the context, e.g. bank (river or money).

2.5 FastText

Allows to figure out some new words. Braking up words into subwords. E.g. <where> -> <wh, whe, her, ere, re> Insufficeint, the "in" has different meaning between two words and the prefixes of a word.

Does this works well? Not the perfect solution.

2.6 Visualizations - Semantic meanings

Reducing to 2 dimensions with Principal Component Analysis (for visualization purposes). The words belonging to the 3 categories are well-separated in the “compressed” embedding space

2.7 Inside an RNN

We have three dense NN layers. Two input and two output.

- **Input layer:** processes the current input, produces a vector with the same dimensionality as the hidden state. Makes the input and the state the same size dimentionality
- **State layer:** processes the hidden state from the previous step
- **Output layer:** Summed the two vectors and produces

2.7.1 Weight sharing

Backpropagation-through-time

2.8 Encoder-decoder architecture

Ideally, the model should:

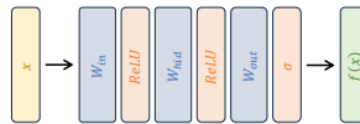
- See the entire sequence before starting generating the output



Capitolo 3

Recurrent Neural Networks

Limitations of Fully-Connected Neural Networks (FCNNs). They have some inputs and some output layers, interleaved by some hidden layers.



This type of models have some architectural constraints

- The input/feature is expected to be fixed in size, because each neuron has a specific set of weights
- Each input/feature is associated with its set of weights
- The output/feature is expected to be fixed in size

The input is expected to be fixed in size

It is ok when it is structured, such as a table. We only manage missing values. For unstructured and dynamic length data, such as images or text.

Each input is associated with its set of weights

If the same pattern occurs in different portions of the input, the model needs to learn the same pattern multiple times.

The output is expected to be fixed in size

As defined by the number of rows in W_{out} . Sometimes, we may want outputs to have varying sizes. E.g. In object detection, we want to segment the pixels of an image

3.1 Recurrent Neural Networks - RNN

Type of NN designed to process sequential data. It receives and looks at one element at a time. And to let the model remember what it has seen before, we add a hidden state to summarize the info seen so far.

RNN architecture

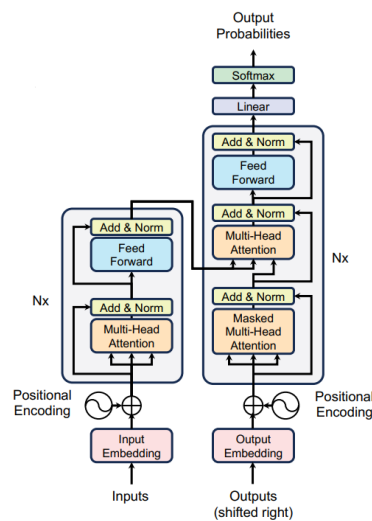
The general RNN architecture is characterized by two **inputs** and two **outputs**:

- The first input is the current element of the sequence
- The second input is the hidden state from the previous time

RNN architecture, unfolded

Capitolo 4

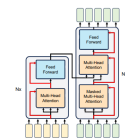
Transformers



Transformers architecture are not longer based on RNN. We shot the model the entire sequence very puerfull, but is expensive.

4.1 Simplified architecture

We need yellow imputes and the model give us the green outputs.



We have two types of inputs:

- FF - Feed forward

4.2 Generation the first token

The input sequence pass to encoder model and the output of the encoder producers a vector for each token provided in input. 10 input 10 output, is a property of the attention layer, this means no compression in a single vector and no loss of information if it is a good encoder.

The decoder takes the output of the encoder and we need to speciify the first token, we give the special vector. The model will look it and decide then it prdouce output 1 input 1 output (BOS). The output is always a probabiilty function in our vocaboly. That word become the second word on the decoder input. The encoder knows the intire previous sequence, at the end we produce a end of sequence token (EOS).

Note: The decoder will output a prediction for each input token. At inference time, we are interested the latest generated token. At training time, the loss is computed on all tokens generated at that step!

Cosality masking, 2 input 2 output, but the first token LLM should not see on the GUI, it already printed.

4.3 Tokenization

We need a way to split a sentence into units. This process is called tokenization, and splits the sequence into tokens

Transformers need to process natural language, in the form of sentences, braking into small units.

Character-level

Mrs. Rucastle was downstairs, so I had an admirable opportunity.

Braking up the sentence into characters. Each character is a token.

Pros:

- Simple solutions
- No out-of-vocabulary problem
- Robust to misspellings and variations

Cons:

- Long sequences - transformers, attention mechanisms, are $O(N^2)$ in the sequence length
- Slower/more complex training
- Semantic information is harder to capture
- Inefficient for common words

Word-level

Breaks text into individual words. Each word is treated as as token

Mrs. Rucastle was downstairs, so I had an admirable opportunity.

Pros:

- Captures semantic meaning directly
- Shorter sequences compared to character-level
- More intuitive/easier to understand

Cons:

- Out-of-vocabulary (OOV) issues for rare or new words
- Does not leverage some information shared among words (e.g. prefixes/suffixes)
- Larger vocabulary needed, leading to memory inefficiency
- Vectors for rarer words are trained “less”

Subword-level

Breaks text into subword units (e.g., prefixes, suffixes). fastText uses n-grams contained in each word. A more commonly used method is Byte-Pair Encoding (BPE)

Mrs. Rucastle was downstairs, so I had an admirable opportunity.

Pros:

- Balances between character-level and word-level
- Handles out-of-vocabulary words effectively
- Compact vocabulary with better generalization
- Efficient for both frequent and rare words

In practice, subword-level is the most commonly adopted tokenization method. A simple, yet commonly adopted technique is BPE - Byte-Pair Encoding.

Cons:

- Common words are represented by fewer subwords w.r.t. rarer words
- Requires defining a subword definition policy
- The subword definition policy may introduce some computational overhead

4.4 Byte-Pair Encoding - desiderata

Data driven approach is used to chose "best" tokens. To do so, we need a colection of text called **corpus**.

- Common words to be encoded into a single token
- Rare words to be encoded into multiple tokens

Also we define the desired maximum number of tokens

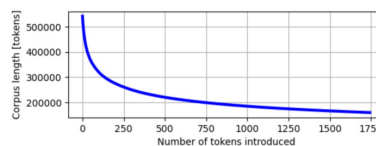
1. The corpus is initially encoded with 1 character byte = 1 token so we address all possible sentences.
2. We count the frequency of each pair of subsequent tokens is counted in the corpus
3. The most frequent pair of tokens (F_1, F_2) is assigned a new token T
4. All occurrences of the pair (F_1, F_2) in the corpus are replaced with T
5. The process 2-4 is repeated until the desired number of tokens is reached

BPE example : The corpus we we would use as example is "would a woodchuck chuck wood"

- Corpus: "would a woodchuck chuck wood"
- Initial encoding: w | o | u | l | d | | a | | w | o | o | d | c | h | u | c | k | | c | h | u | c | k | | w | o | o | d
- Count frequency of pairs: (w, o): 3, (o, o): 2 ...
- Assign a new token to the most frequent pair: (w, o) $\rightarrow$ wo

[illegible]

4.4.1 BPE results



The number of tokens used to represent the corpus decreases as we increase the number of tokens used. There is a reasonable small number of tokens that can be used to represent a large corpus.

Typically, 10’s to 100’s of thousands of tokens are used

4.5 Some extra tokens

We introduce some special tokens to indicate that special positions within a sequence. For instance, we can use tokens for beginning/end of sequence during training.

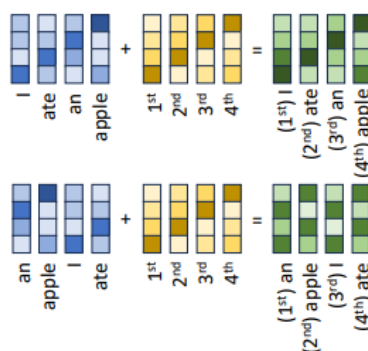
- BOS - Beginning of Sequence
- EOS - End of Sequence
- CLS - Classification token for task-specific use
- SEP - Separator token within the same input. E.g. Separate Question from Answer in Question Answering tasks

4.6 Positional encoding

Every time we have a token is associated to a vector learned during the training, start with random vectors, and adjust them via gradient descent.

The **same token** is mapped to the **same vector** regardless of its position in the sequence. This is not ideal. Additionally the **Attention and Feed Forward does not understand the order** of the elements.

We have to figure out a way to inject the position before passing it to the transformer. This is called **positional encoding**.



4.7 Sinusoidal Positional encoding

In the AIAYN paper, the PE vectors are defined as:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

